Notes on Machine Learning

Course of Machine Learning

Master Degree in Computer Science

University of Rome "Tor Vergata"

Giorgio Gambosi

Academic Year: 2025-2026

Chapter 1

Loss functions & training: A Summary

Loss function

- The loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}$ quantifies the **cost** of predicting y instead of the true target t.
- For an input $\mathbf{x}$ with true output t, the prediction quality is measured by:

$$\mathcal{R}(\mathbf{x}, t) = L(h(\mathbf{x}), t) \tag{1.1}$$

- The **empirical risk** aggregates loss over the training set:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(x,t)\in\mathcal{T}} L(h(x), t) \tag{1.2}$$

- During training, we work with parametric functions $h = h_\theta$, where $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_d)$.
- Training minimizes the overall loss:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}) \tag{1.3}$$

  where $L_i = L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$ for each data point.

Loss function minimization approaches

Global Minimum

- A **global minimum** is a point where the loss is lower than at any other point in parameter space.
- At any minimum, the gradient must be zero:

$$\nabla\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \mathbf{0} \tag{1.4}$$

- The gradient ∇f is the vector of partial derivatives:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_m} \end{pmatrix} \tag{1.5}$$

- Setting gradient to zero means solving:

$$\frac{\partial}{\partial \theta_i} \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = 0 \quad \forall i \tag{1.6}$$

- **Challenges:**
  - System has multiple solutions (minima, maxima, saddle points).
  - Analytical solutions often impossible for complex models.

Gradient descent

- Iterative numerical method to find a local minimum.
- **Principle:** Move in the direction opposite to the gradient (steepest descent).
- Start from initial parameters $\boldsymbol{\theta}^{(0)}$ with initial error $\overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta}^{(0)})$.
- **Update rule:**

$$\theta_j^{(i)} := \theta_j^{(i-1)} - \eta \frac{\partial}{\partial \theta_j} \overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}^{(i-1)}} = \theta_j^{(i-1)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial}{\partial \theta_j} L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)\Big|_{\boldsymbol{\theta}^{(i-1)}}, \tag{1.7}$$

- In matrix form:

$$\boldsymbol{\theta}^{(i)} := \boldsymbol{\theta}^{(i-1)} - \eta \, \nabla \overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}^{(i-1)}} = \boldsymbol{\theta}^{(i-1)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)\Big|_{\boldsymbol{\theta}^{(i-1)}}. \tag{1.8}$$

- η is the **learning rate** controlling step size.
- **Key challenges:**
  - Local minima vs global minimum.
  - Saddle points.
  - Convergence speed.
  - Learning rate selection.

Convexity

- A set $S \subset \mathbb{R}^d$ is **convex** if for any $\mathbf{x}_1, \mathbf{x}_2 \in S$ and $\lambda \in (0, 1)$:

$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S \tag{1.9}$$

- A function $f(\mathbf{x})$ is **convex** if for all $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in (0, 1)$:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2) \tag{1.10}$$

- **Strict convexity** requires strict inequality for $\mathbf{x}_1 \neq \mathbf{x}_2$.
- **Key property:** For convex functions, any local minimum is also a global minimum.
- For strictly convex functions, there is exactly one local minimum (the global minimum).
- Properties:
  1. Sum of (strictly) convex functions is (strictly) convex.
  2. Product of (strictly) convex function and positive constant is (strictly) convex.
- **Implications for empirical risk:**
  - If each $L(\boldsymbol{\theta}; \mathbf{x}, t)$ is convex, empirical risk is convex.
  - Any local minimum of empirical risk is global.
  - If strictly convex, unique minimum exists.

Some common loss functions

Loss functions for regression

Quadratic loss

- Definition: $L(y, t) = (y - t)^2$
- Empirical risk: $\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} (h(\mathbf{x}) - t)^2$ (Mean Squared Error)
- For linear regression: $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$, loss becomes:

$$\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)^2 \tag{1.11}$$

- **Properties:**
  - Strictly convex $\Rightarrow$ unique global minimum.
  - Gradient is linear, making optimization tractable:

$$\frac{\partial}{\partial w_i}\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)x_i$$
$$\frac{\partial}{\partial b}\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)$$

  - Closed-form solution exists (normal equations).
  - **Drawback:** Not robust to outliers (errors squared $\Rightarrow$ large errors dominate).

Absolute loss

- Definition: $L(y, t) = |y - t|$ (L1 loss, Mean Absolute Error)
- **Properties:**

- Penalizes errors linearly, more robust to outliers.
- Convex but not strictly convex (flat gradient regions).
- Gradient is piecewise constant (-1 when $y < t$, $+1$ when $y > t$).
- Gradient undefined at $y = t$ (requires subgradients).

Huber loss

- Definition (with parameter δ):

$$L(t, y) = \begin{cases} \frac{1}{2}(t - y)^2 & |t - y| \leq \delta \\ \delta(|t - y| - \frac{\delta}{2}) & |t - y| > \delta \end{cases} \tag{1.12}$$

- Combines quadratic behavior near zero (smooth gradients) with linear growth for large errors (robustness to outliers).
- δ controls transition point.

Loss functions for classification

Classification approaches

- Two approaches:
  - **Direct class prediction:** Output class label directly.
  - **Probabilistic prediction:** Output probability distribution over classes.
- Focus on binary classification with $t \in \{-1, +1\}$, prediction $y = h(\mathbf{x})$ (confidence score, sign determines class).

0/1 loss

- Definition:

$$L(t, y) = \begin{cases} 1 & \text{sgn}(y) \neq t \\ 0 & \text{sgn}(y) = t \end{cases} \tag{1.13}$$

- Compact form: $L(t, y) = \mathbb{1}[ty < 0]$
- **Problems:**
  - Non-convex.
  - Non-smooth (discontinuous at $ty = 0$).
  - Zero gradients almost everywhere $\Rightarrow$ no gradient information.
  - Minimization is NP-hard.

Convex surrogate loss functions

- Surrogate loss substitutes for 0/1 loss during optimization.
- Required properties:
  - **Upper bound property:** $L_{\text{surrogate}}(t, y) \geq \mathbb{1}[ty < 0]$
  - **Convexity:** Guarantees global minima.
  - **Smoothness:** Enables gradient-based optimization.

Perceptron loss

- Definition: $L(t, y) = \max(0, -ty)$
- **Properties:**
  - Zero for correctly classified elements ($ty > 0$).
  - For misclassified elements ($ty < 0$), loss $= -ty = |ty|$ (penalizes confident wrong predictions more).
  - Continuous everywhere.
  - Gradient continuous almost everywhere (except $ty = 0$).
  - Convex (not strictly convex).
  - **Not** a strict surrogate (doesn't always upper bound 0/1 loss).

Hinge loss

- Definition: $L(t, y) = \max(0, 1 - ty)$
- **Key differences from perceptron loss:**
  - Loss zero only when $ty \geq 1$ (requires confident correct predictions).
  - For $0 < ty < 1$, loss $= 1 - ty > 0$ (penalizes correct but weak predictions).
  - For misclassified examples ($ty < 0$), loss $= 1 - ty$ (grows linearly).
- **Properties:**
  - Continuous everywhere.
  - Gradient continuous almost everywhere (except $ty = 1$).
  - Convex (not strictly convex due to flat region $ty > 1$).
  - Proper surrogate loss (upper bounds 0/1 loss).
  - Used in Support Vector Machines (encourages margin maximization).

Subgradients

- Handle non-differentiability at singular points.
- A subgradient $\overline{\nabla} f|_x$ satisfies:

$$f(x') \geq f(x) + \overline{\nabla} f|_x \cdot (x' - x) \quad \text{for all } x' \tag{1.14}$$

- At non-differentiable points, multiple subgradients exist.
- For hinge loss at $ty = 1$, valid subgradients are any slope in $[-t, 0]$ (when $t = 1$) or $[0, -t]$ (when $t = -1$).
- Common choice: $\frac{\partial L_H}{\partial y} = 0$ at $ty = 1$, giving:

$$\overline{\nabla}_y L_H = \begin{cases} -t & ty < 1 \\ 0 & ty \geq 1 \end{cases} \tag{1.15}$$

Square loss in classification

- Definition: $L(t, y) = (1 - ty)^2$
- **Properties:**
  - Continuous and infinitely differentiable.
  - Gradient continuous everywhere.
  - Convex.
  - **Drawbacks:**
    * Not a surrogate loss (doesn't upper bound 0/1 loss everywhere).
    * Extreme errors heavily penalized (quadratic).
    * Symmetric penalization: even confident correct predictions ($ty \gg 1$) are penalized.

Log loss (cross entropy)

- Definition: $L(t, y) = \frac{1}{\log 2} \log(1 + e^{-ty})$ (normalization factor sometimes omitted)
- Smoothed version of hinge loss.
- **Properties:**
  - Continuous and infinitely differentiable everywhere.
  - Gradient continuous at all points.
  - Strictly convex $\Rightarrow$ unique global minimum.
  - Proper surrogate loss (upper bounds 0/1 loss).
  - As $ty \to -\infty$, loss grows linearly.
  - As $ty \to +\infty$, loss approaches zero asymptotically.
- Central to logistic regression and probabilistic classification.

Connection to information theory

- **Cross entropy:** $\mathbb{H}(p, q) = -\mathbb{E}_{x \sim p}[\log q(x)]$
- **KL divergence:** $D_{KL}(p \| q) = \mathbb{H}(p, q) - \mathbb{H}(p)$ (non-negative, zero iff $p = q$)
- **Entropy:** $\mathbb{H}(p) = -\mathbb{E}_{x \sim p}[\log p(x)]$ (minimum average bits needed)

- For binary classification with true labels $t \in \{0, 1\}$ and predicted probabilities $y(\mathbf{x}) \in [0, 1]$:

$$\mathrm{CE}(\mathcal{T}) = -\frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \Big(t \log y(\mathbf{x}) + (1-t)\log(1-y(\mathbf{x})) \Big) \tag{1.16}$$

- For logistic regression ($y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}}$) and using $t \in \{-1, +1\}$:

$$\mathrm{CE}(\mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}) \tag{1.17}$$

- This matches log loss empirical risk (up to constant $\log 2$):

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}| \log 2} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}) \tag{1.18}$$

Exponential loss

- Definition: $L(t, y) = e^{-ty}$
- Used in boosting algorithms (AdaBoost).
- **Properties:**
  - Continuous and infinitely differentiable.
  - Gradient continuous everywhere.
  - Strictly convex.
  - Proper surrogate loss.
  - Penalizes errors more aggressively than log loss (exponential growth).
  - Very sensitive to outliers.

Computing h^*

- Minimization of empirical risk over parameterized hypothesis class ($\theta \in \Theta = \mathbb{R}^d$) is an unconstrained optimization problem.
- Necessary condition for optimality: stationary point where gradient vanishes:

$$\frac{\partial}{\partial \theta_i} \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = 0 \tag{1.19}$$

  or in vector form:

$$\nabla \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = \mathbf{0} \tag{1.20}$$

- Analytical solution generally difficult or infeasible ⇒ numerical optimization techniques.

Gradient descent

- Minimize differentiable function $J(\theta)$ by iteratively moving opposite to gradient.
- **Update rule:**

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla J|_{\theta^{(k)}}$$

- Component-wise:

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{1.21}$$

- Four-step formulation:

$$\begin{aligned} g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \\ \eta_{j,k} &= \eta \\ \Delta_{j,k} &= -\eta_{j,k}\, g_{j,k} \\ \theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k} \end{aligned}$$

- Learning rate η controls step size; critical for convergence speed and stability.

Batch gradient descent

- Objective: empirical risk over entire training set:

$$J(\theta) = \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h_\theta(\mathbf{x}), t) \tag{1.22}$$

- Gradient:

$$g_{j,k} = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{1.23}$$

- Update:

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{1.24}$$

- **Example (linear regression):**

$$\begin{aligned} \theta_j^{(k+1)} &= \theta_j^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j \quad j = 1, \dots, d \\ \theta_0^{(k+1)} &= \theta_0^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) \end{aligned}$$

- **Properties:**
  - Smooth, stable decrease of empirical risk.
  - Computationally expensive for large datasets.
  - Guaranteed convergence to global minimum if convex, local minimum if non-convex.
- Pseudocode:

```
for i in range(nb_epochs):
  params_grad = evaluate_gradient(loss_function, data, params)
  params = params - learning_rate * params_grad
```

Stochastic gradient descent (SGD)

- Update using a single training example at a time:

$$J(\theta) = L(h_\theta(\mathbf{x}), t) \tag{1.25}$$

- Gradient:

$$g_{j,k} = \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{1.26}$$

- Update:

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \frac{\partial}{\partial \theta_j} L(h_\theta(\mathbf{x}), t)\bigg|_{\theta^{(k)}} \tag{1.27}$$

- **Properties:**
  - Much faster updates, supports online learning.
  - High variance in updates (noisy trajectory).
  - Stochasticity can help escape shallow local minima or saddle points.
  - With decreasing learning rate, converges almost surely to local minimum.
- Pseudocode:

```
for i in range(nb_epochs):
  np.random.shuffle(data)
  for example in data:
    params_grad = evaluate_gradient(loss_function, example, params)
    params = params - learning_rate * params_grad
```

- **Linear regression example:**

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j$$
$$\theta_0^{(k+1)} = \theta_0^{(k)} - \eta \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right)$$

Mini-batch gradient descent

- Intermediate between batch and SGD: use mini-batch B of size m.
- Objective over mini-batch:

$$J(\theta) = \frac{1}{m} \sum_{(\mathbf{x},t)\in B} L(h_\theta(\mathbf{x}), t) \tag{1.28}$$

- Gradient:

$$g_{j,k} = \frac{1}{m} \sum_{(\mathbf{x},t)\in B} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\Big|_{\theta^{(k)}} \tag{1.29}$$

- Update:

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in B} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\Big|_{\theta^{(k)}} \tag{1.30}$$

- **Properties:**
  - Limits computational cost per update.
  - Reduces variance compared to SGD, more stable convergence.
  - Computation can be parallelized on GPUs.
  - Mini-batch size m is a hyperparameter (typical values 32-256).
  - Standard choice for training neural networks.
- Pseudocode:

```
for i in range(nb_epochs):
  np.random.shuffle(data)
  for batch in get_batches(data, batch_size=m):
    params_grad = evaluate_gradient(loss_function, batch, params)
    params = params - learning_rate * params_grad
```

- **Linear regression example:**

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j$$
$$\theta_0^{(k+1)} = \theta_0^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right)$$

Open issues

- Learning rate selection: too small ⇒ slow convergence; too large ⇒ instability/divergence.
- Learning rate schedules help but require predefined hyperparameters.
- Single global learning rate applied to all parameters may be suboptimal.
- Non-convex problems have many local minima and saddle points; saddle points surrounded by flat regions cause slow progress.

Momentum gradient descent

- Physical analogy: parameter vector θ as position of a body of unit mass.
- Force: $F(\theta) = -\eta \nabla J(\theta)$ (negative gradient as acceleration).
- Update governed by velocity, not directly by acceleration:

$$\theta_j^{(k+1)} = \theta_j^{(k)} + v_j^{(k+1)}$$
$$v_j^{(k+1)} = v_j^{(k)} - \eta_{j,k}\, g_{j,k}$$

- With $v_j^{(0)} = 0$ and constant learning rate:

$$\theta_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=0}^{k} g_{j,r} \tag{1.31}$$

- Practical version with dampening (friction) parameter $\gamma \in [0, 1)$:

$$v_j^{(k+1)} = \gamma v_j^{(k)} - \eta g_{j,k}$$
$$\theta_j^{(k+1)} = \theta_j^{(k)} + v_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=1}^{k} \gamma^{k-r} g_{j,r}$$

- **Properties:**
  - Uses weighted sum of past gradients.
  - Velocity increases along directions with consistent gradients.

  - Oscillations dampened along directions with rapidly changing gradients.
  - Reduces fluctuations, enables faster and more stable convergence.

- **Linear regression example (mini-batch):**

$$
\begin{aligned}
v_j^{(k+1)} &= \gamma v_j^{(k)} - \eta \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(k)} x_s + \theta_0^{(k)} - t \right) x_j \\
v_0^{(k+1)} &= \gamma v_0^{(k)} - \eta \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(k)} x_s + \theta_0^{(k)} - t \right) \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + v_j^{(k+1)}
\end{aligned}
$$

Nesterov accelerated gradient descent (NAG)

- Uses look-ahead estimate $\tilde{\theta}^{(k+1)} \triangleq \theta^{(k)} + \gamma v^{(k)}$ before computing gradient.
- Update equations:

$$
\begin{aligned}
\tilde{\theta}_j^{(k+1)} &= \theta_j^{(k)} + \gamma v_j^{(k)} \\
v_j^{(k+1)} &= \gamma v_j^{(k)} - \eta g_{j,k+1} \\
\theta_j^{(k+1)} &= \tilde{\theta}_j^{(k)} + v_j^{(k+1)}
\end{aligned}
$$

- **Properties:**
  - Gradient evaluated at approximated future position.
  - Anticipates changes in velocity, correcting trajectory earlier.
  - More responsive and often faster convergence than standard momentum.

Dynamically updating the learning rate

- Fixed learning rate often suboptimal: larger steps early, smaller steps near optimum.
- **Learning rate schedules:**
  - Step decay: reduce by factor c every K steps.
  - Exponential decay: $\eta^{(k)} = \eta^{(0)} e^{-\alpha k}$.
  - $1/t$ decay: $\eta^{(k)} = \frac{\eta^{(0)}}{1+\alpha k}$.
- Limitations: hyperparameters problem-dependent; same rate applied to all parameters.
- Adaptive methods assign parameter-specific learning rates based on gradient history.

Adagrad

- Parameter-specific learning rates that adapt to historical gradient magnitudes.

- For parameter θ_j at step k:

$$
\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \\
G_{j,k} &= G_{j,k-1} + g_{j,k}^2 = \sum_{i=0}^{k} g_{j,i}^2 \\
\eta_{j,k} &= \frac{\eta}{\sqrt{G_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}
\end{aligned}
$$

- ε small constant for numerical stability.
- **Properties:**
  - Larger updates for rarely updated parameters; smaller updates for frequently updated ones.
  - Well-suited for sparse data or heterogeneous features.
  - Learning rate decays monotonically $\Rightarrow$ eventually becomes too small, halting learning.

RMSprop

- Addresses Adagrad's monotonic decay by using exponentially decaying average of squared gradients.
- Decaying cumulative gradient:

$$
\tilde{G}_{j,k} = \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 = (1-\gamma) \sum_{i=0}^{k} \gamma^{k-i} g_{j,i}^2
$$

- Learning rate:

$$
\eta_{j,k} = \frac{\eta}{\sqrt{\tilde{G}_{j,k} + \varepsilon}} \tag{1.32}
$$

- Update:

$$
\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 \\
\eta_{j,k} &= \frac{\eta}{\sqrt{\tilde{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}
\end{aligned}
$$

- γ controls decay rate (typically $\gamma \simeq 0.9$).
- **Properties:** Learning rate adapts mainly to recent gradient magnitudes; allows continued learning.

Adadelta

- Eliminates need for global learning rate η.
- Uses decaying average of past squared updates:

$$\overline{G}_{j,k} = \gamma \overline{G}_{j,k-1} + (1-\gamma)\Delta_{j,k}^2 = (1-\gamma)\sum_{i=0}^{k} \gamma^{k-i}\Delta_{j,i}^2$$

- Update rule:

$$\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}} \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 \\
\eta_{j,k} &= -\frac{\sqrt{\overline{G}_{j,k-1} + \varepsilon}}{\sqrt{\tilde{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\overline{G}_{j,k} &= \gamma \overline{G}_{j,k-1} + (1-\gamma)\Delta_{j,k}^2 \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}
\end{aligned}$$

- $\gamma \simeq 0.95$, $\varepsilon \simeq 10^{-8}$.
- Ensures units of update $\Delta_{j,k}$ consistent with those of parameter θ_j.

Adam (Adaptive Moment Estimation)

- Combines RMSprop with momentum (exponentially decaying sum of gradients).
- Momentum term:

$$\tilde{H}_{j,k} = \beta \tilde{H}_{j,k-1} + (1-\beta) g_{j,k} = (1-\beta)\sum_{i=0}^{k} \beta^{k-i} g_{j,i} \tag{1.33}$$

- Bias correction (compensates for initialization at zero):

$$\begin{aligned}
\hat{G}_{j,k} &= \frac{\tilde{G}_{j,k}}{1-\gamma^k} \\
\hat{H}_{j,k} &= \frac{\tilde{H}_{j,k}}{1-\beta^k}
\end{aligned}$$

- Complete update:

$$
\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}} \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 \\
\tilde{H}_{j,k} &= \beta \tilde{H}_{j,k-1} + (1-\beta) g_{j,k} \\
\hat{G}_{j,k} &= \frac{\tilde{G}_{j,k}}{1-\gamma^k} \\
\hat{H}_{j,k} &= \frac{\tilde{H}_{j,k}}{1-\beta^k} \\
\eta_{j,k} &= \frac{\eta}{\sqrt{\hat{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} \hat{H}_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}
\end{aligned}
$$

- Default values: $\eta \simeq 0.001$, $\gamma \simeq 0.9$, $\beta \simeq 0.999$, $\varepsilon \simeq 10^{-8}$.
- Widely used due to robustness and good empirical performance.

Reassuming table

Method	$G^{(k)}$	$\eta^{(k)}$
GD	$g^{(k)}$	η
Adagrad	$g^{(k)}$	$\frac{\eta}{RSS(g^2)}$
RMSprop	$g^{(k)}$	$\frac{\eta}{RDSS(g^2,\gamma)}$
Adadelta	$g^{(k)}$	$\frac{RDSS(\Delta^2,\gamma)}{RDSS(g^2,\gamma)}$
Adam	$RDSS(g,\beta)$	$\frac{\eta}{RDSS(g^2,\gamma)}$

$$
\begin{aligned}
\Delta^{(k)} &= -\eta^{(k)} G^{(k)} \\
\theta_{k+1} &= \theta^{(k)} + \Delta^{(k)}
\end{aligned}
$$

where:

$$
\begin{aligned}
RSS(f) &= \sqrt{f^{(0)} + f^{(1)} + \cdots + f^{(k-1)} + f^{(k)} + \varepsilon} \\
RDSS(f,\gamma) &= \sqrt{(1-\gamma)(\gamma^k f^{(0)} + \gamma^{k-1} f^{(1)} + \cdots + \gamma f^{(k-1)} + f^{(k)}) + \varepsilon}
\end{aligned}
$$

Second-Order Methods

- Seek points where $\nabla J(\theta) = 0$ using Newton-Raphson method on gradient.
- One-dimensional Newton:

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{J'(\theta)}{J''(\theta)}\bigg|_{\theta^{(k)}} \tag{1.34}$$

- Multi-dimensional Newton:

$$\theta^{(k+1)} = \theta^{(k)} - \left(H(J)^{-1}\nabla J\right)\Big|_{\theta^{(k)}} \tag{1.35}$$

  where $H(J)_{ij} = \frac{\partial^2 J}{\partial\theta_i\partial\theta_j}$ is the **Hessian matrix**.
- **Geometric interpretation:**
  - Gradient Descent: approximates $J(\theta)$ with tangent plane, moves in steepest descent direction.
  - Newton's method: approximates $J(\theta)$ with quadratic surface, moves to its minimum.
- With damping factor $\eta < 1$:

$$\Delta^{(k)} = -\left(H(J)^{-1}\nabla J\right)\Big|_{\theta^{(k)}}$$
$$\theta^{(k+1)} = \theta^{(k)} + \eta\Delta^{(k)}$$

- **Computational drawbacks:**
  - Memory: storing Hessian requires $O(n^2)$ space (prohibitive for high dimensions).
  - Computation: inverting Hessian or solving $H\Delta = -\nabla J$ scales as $O(n^3)$.
- **Quasi-Newton methods** (BFGS, L-BFGS) approximate inverse Hessian using gradient information.
  - BFGS: $O(n^2)$ computation, $O(n^2)$ memory.
  - L-BFGS: $O(mn)$ computation, $O(mn)$ memory with $m \ll n$ (linear complexity).
  - Achieve superlinear convergence without explicit second derivatives.

Optimization in deep learning

Why first-order methods dominate

- **Scale:** Modern neural networks have billions of parameters; Hessian would have n^2 entries (impossible to store or invert).
- **Memory efficiency:** Adam maintains only two decaying sums per parameter ($O(n)$ memory).
- **Noise robustness:** Deep learning uses mini-batches; second-order methods are sensitive to noise in gradient estimates. Adam's decaying sums act as low-pass filters, smoothing noisy gradients.
- **Saddle points:** In high-dimensional spaces, saddle points are more common than local minima.
  - Newton's method targets zero-gradient points, can be attracted to saddle points.

- Gradient descent naturally moves away from saddle points by following downward slopes.
- Decomposing Hessian into eigenvectors q_i and eigenvalues λ_i, Newton update in direction i is $\Delta\theta_i = -\frac{1}{\lambda_i} g_i$.
- With negative curvature ($\lambda_i < 0$), Newton moves *with* the gradient, climbing toward the saddle point.